

A5 Consultas Federadas con Trino

Motor distribuido sobre fuentes heterogéneas

Asignatura	Big data avanzado
Autores	Adrián Blasco Lozano · Antonio Sala Llaudis · Enric Gil Baquero · Hugo Romero Casado
Grupo	10
Institución	Universitat Oberta de Catalunya
Fecha	14/05/2026
Repositorio	github.com/blasketch/big_data_a5
Vídeo (YouTube)	https://youtu.be/dRQJXjHWNL4



Universitat Oberta
de Catalunya

1. Estrategia de despliegue

Esta actividad despliega un motor de consultas distribuido capaz de cruzar datos residentes en sistemas heterogéneos sin desplazarlos físicamente. El objetivo es demostrar la capacidad de un motor federado para actuar como punto único de acceso SQL sobre fuentes de naturaleza distinta, permitiendo realizar JOINS entre datos relacionales y un Data Lake en formato Parquet en una única sentencia SQL, sin necesidad de ETL previo ni almacén intermedio.

1.1 Motor elegido: Trino 451

Hemos optado por Trino 451 en lugar de PrestoDB. Trino es la evolución mantenida del proyecto PrestoDB original, comparte la misma arquitectura Coordinator/Worker y la misma sintaxis SQL, pero aporta mejor soporte multiplataforma, imágenes Docker actualizadas y la configuración adicional de JVM necesaria para equipos sin soporte x86-64-v3.

1.2 Caso de negocio

El sistema modela el cuadro de mando de una empresa con dos fuentes de datos en sistemas distintos:

- **PostgreSQL (OLTP transaccional):** tabla clientes con 1.000 registros (identificador, nombre, país, score crediticio). Representa los datos maestros de gestión.

- **Data Lake en Parquet (analítico):** tabla ventas_historicas con 100.000 registros (id_venta, id_cliente, monto, fecha). Representa el histórico de operaciones.

El KPI federado calcula la facturación total, el número de operaciones y el ticket medio por país, cruzando ambas fuentes en una única sentencia SQL sin mover los datos.

2. Arquitectura y configuración técnica

2.1 Diagrama de la arquitectura desplegada

La topología completa se levanta con un único docker-compose.yml. Los cuatro servicios comparten la red Docker dedicada trino-net, separando cómputo (Coordinator/Workers) de almacenamiento (PostgreSQL y el volumen datalake):

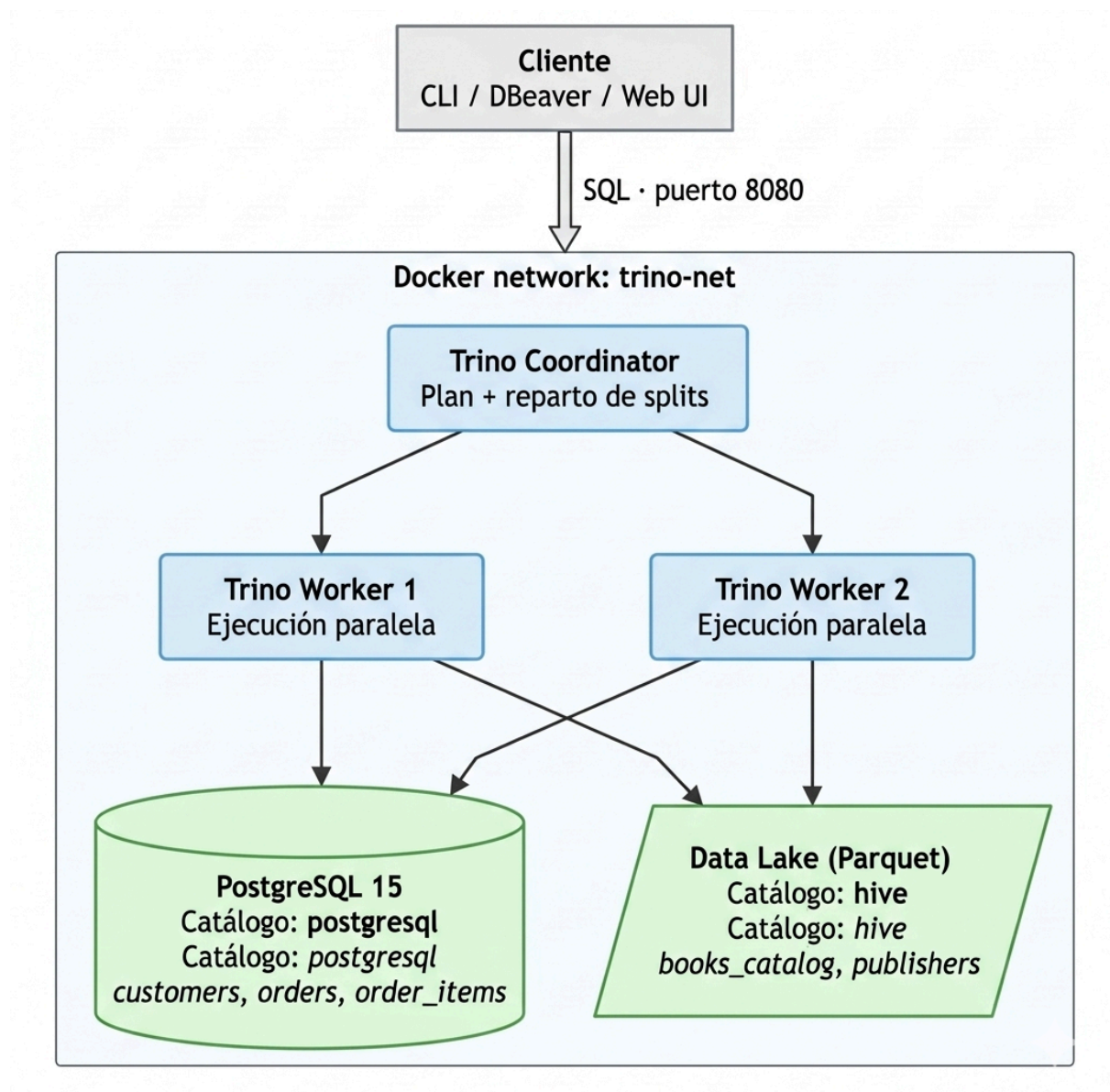


Figura 1. Topología del clúster Trino federado (exportar desde docs/architecture.md vía mermaid.live).

2.2 Componentes del clúster

Componente	Imagen / Config.	Rol en el sistema
Trino Coordinator	trinodb/trino:451 JVM 1 GB puerto 8080	Cerebro del clúster: recibe las queries SQL, genera el plan de ejecución, lo optimiza y reparte los splits entre los Workers. No ejecuta cómputo directamente.
Trino Worker 1 y 2	trinodb/trino:451 JVM 1 GB cada uno	Músculo del clúster: ejecutan los splits en paralelo, leen directamente de las fuentes de datos y devuelven resultados parciales al Coordinator.
PostgreSQL 15	postgres:15 puerto 5432 BD: base_datos_grupo	Fuente relacional transaccional (OLTP). Aloja la tabla clientes. Accedida por Trino vía el catálogo postgresql con predicate pushdown.
Volumen ./datalake	Filesystem local /opt/datalake en 3 nodos	Data Lake simulado. Carpeta del host montada en los tres nodos Trino para permitir lectura paralela sin duplicar datos. Accedida vía el catálogo hive.

Tabla 1. Componentes del clúster Trino con su configuración y rol en el sistema.

2.3 Conectores configurados y su propósito

Trino actúa como motor de consultas unificado. Hemos habilitado dos conectores, desplegados de forma idéntica en coordinator/etc/catalog/, worker1/etc/catalog/ y worker2/etc/catalog/ para garantizar que cualquier Worker pueda resolver cualquier consulta de forma autónoma:

Catálogo	Tipo de conector	Propósito en el negocio
postgresql	Conector JDBC relacional connection-url: jdbc:postgresql://postgres-db:5432/base_datos_grupo	Gestiona la capa relacional transaccional. Contiene la información maestra de clientes. Soporta predicate pushdown e índices de PostgreSQL.

hive	Conector Hive con metastore basado en filesystem hive.metastore=file catalog.dir=/opt/data lake	Mapea y consulta archivos externos en formato Parquet almacenados en el Data Lake local, exponiéndolos como tablas SQL. Permite análisis sobre datos históricos distribuidos.
------	--	---

Tabla 2. Conectores Trino configurados, su tipo y propósito en el sistema.

2.4 Infraestructura como código

Todo el stack se levanta con un único comando. El script de generación de datos (semilla 42) garantiza reproducibilidad completa:

```
# 1. Generar datasets reproducibles (seed=42)
```

```
python script/generar_dataset.py
```

```
# 2. Levantar el clúster completo
```

```
docker compose up -d
```

3. Consulta federada y análisis de rendimiento

3.1 Diseño de la consulta SQL

La consulta federada cruza las dos fuentes mediante `id_cliente`, aplica predicate pushdown vía el filtro `WHERE`, y agrega resultados con `SUM`, `COUNT` y `AVG`

SELECT
c.pais,
COUNT(v.id_venta) AS total_operaciones,
ROUND(SUM(v.monto), 2) AS facturacion_total,
ROUND(AVG(v.monto), 2) AS media_operacion

FROM hive.default.ventas_historicas v
JOIN postgresql.public.clientes c
ON v.id_cliente = c.id_cliente
WHERE c.pais = 'España'
GROUP BY c.pais;

Código 1. Consulta federada principal. Cruza hive y postgresql en una única sentencia SQL.

3.2 Historial de ejecución Trino Web UI

La siguiente captura muestra el historial real de ejecución en la Trino Web UI (localhost:8080), incluyendo la consulta federada principal, variantes con EXPLAIN ANALYZE y consultas de validación individual de cada fuente. Todas las queries aparecen con estado FINISHED:

QUERY DETAILS			
User, source, query ID, query state, resource group, error name, or query text		State:	Running Queued Finished Failed Sort Reorder Interval Show
20260512_152826_00011_h8x2v	5:28pm	FINISHED	
trino trino-cli global ✓ 162 ▶ 0 0 ⌚ 494.41ms ⌚ 531.93ms ⌚ 237.00ms 📄 0B 🔥 4.52MB 📊 1.34M	SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion FROM hive.default.ventas_historicas v ...		
20260512_152715_00010_h8x2v	5:27pm	FINISHED	
trino trino-cli global ✓ 179 ▶ 0 0 ⌚ 716.38ms ⌚ 785.77ms ⌚ 219.00ms 📄 0B 🔥 5.40MB 📊 784K	EXPLAIN ANALYZE SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion ...		
20260512_152619_00009_h8x2v	5:26pm	FINISHED	
trino trino-cli global ✓ 197 ▶ 0 0 ⌚ 890.62ms ⌚ 1.20s ⌚ 299.00ms 📄 0B 🔥 5.40MB 📊 1.17M	EXPLAIN ANALYZE SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion ...		
20260512_152539_00008_h8x2v	5:25pm	FINISHED	
trino trino-cli global ✓ 1 ▶ 0 0 ⌚ 219.19ms ⌚ 581.26ms ⌚ 0.00ms 📄 0B 🔥 5.17KB 📊 0	EXPLAIN SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion ...		
20260512_152438_00007_h8x2v	5:24pm	FINISHED	
trino trino-cli global ✓ 197 ▶ 0 0 ⌚ 2.10s ⌚ 2.16s ⌚ 1.31s 📄 0B 🔥 5.40MB 📊 3.36M	SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion FROM hive.default.ventas_historicas v ...		
20260512_152408_00006_h8x2v	5:24pm	FINISHED	
trino trino-cli global ✓ 18 ▶ 0 0 ⌚ 1.89s ⌚ 2.18s ⌚ 478.00ms 📄 0B 🔥 2.11KB 📊 1.51K	SELECT * FROM hive.default.ventas_historicas LIMIT 10		
20260512_152405_00005_h8x2v	5:24pm	FINISHED	

Figura 2. Trino Web UI: historial de queries ejecutadas con éxito (estado FINISHED). Los datos de timing son reales.

3.3 Análisis del rendimiento: predicate pushdown

Para evidenciar empíricamente el efecto del predicate pushdown se ejecutaron dos variantes de la consulta: con filtro WHERE c.pais = 'España' (que Trino delega al índice de PostgreSQL antes del JOIN) y sin filtro, que obliga al motor a materializar todos los clientes. Los datos reales extraídos de la Trino UI (12/05/2026) son:

Variante de consulta	Wall time	Splits totales	Filas procesadas
Con pushdown (WHERE pais = 'España')	494 ms	162	1,34 M
Sin pushdown (sin filtro de pais, todos los paises)	2,10 s	197	3,36 M
Mejora obtenida	≈ 4,3x	18% menos	60% menos filas

Tabla 3. Comparativa de rendimiento con y sin predicate pushdown. Datos reales de la Trino UI.

El predicate pushdown reduce el wall time en un factor 4,3x y el volumen de filas procesadas en un 60%, al delegar el filtro pais = 'España' al índice de PostgreSQL antes de que los datos crucen la red interna hacia los Workers. La diferencia en splits (162 vs 197) refleja la poda de particiones del conector Hive para el subconjunto de datos relevantes.

4. Plan de ejecución y análisis de splits

Mediante EXPLAIN ANALYZE se obtiene el plan de ejecución distribuido de la consulta. Los resultados de dos ejecuciones consecutivas (queries 00010 y 00009) muestran:

Parámetro	EXPLAIN ANALYZE #1	EXPLAIN ANALYZE #2
Wall time	785 ms	890 ms
CPU time	219 ms	299 ms
Splits generados	179	197
Datos leídos	5,40 MB	5,40 MB
Filas procesadas	784 K	1,17 M

Tabla 4. Métricas de EXPLAIN ANALYZE. Datos reales del historial de la Trino UI.

El plan de ejecución distribuido se divide en stages interconectados. Trino genera un TableScan sobre el conector PostgreSQL con el predicado pais = 'España' ya empujado al origen, y un TableScan independiente sobre el conector Hive que lee los ficheros Parquet del volumen /opt/datalake. Al ser clientes la tabla pequeña, unas 200 filas tras el filtro y ventas_historicas la tabla grande, con 100.000 filas, Trino selecciona automáticamente un

broadcast join: envía la tabla pequeña a todos los Workers para evitar un shuffle costoso entre nodos. La fase final de agregación es distribuida: cada Worker calcula los SUM y COUNT parciales de su subconjunto, y el Coordinator combina únicamente los resultados ya pre-agregados.

5. Pensamiento critico: impacto de la latencia geografica

¿Como afectaria a la latencia de red si una de las fuentes de datos estuviera en una region geografica distinta?

Desplegar un sistema federado en el que el cómputo y el almacenamiento residen en distintas regiones introduce desafíos severos que van más allá del simple incremento de latencia:

Incremento exponencial del Round-Trip Time (RTT). Al salir de la red local, el RTT pasa de microsegundos (LAN Docker) a 100-150 ms en una conexión transatlántica. Cada fetch de datos desde los Workers hacia la fuente acumula ese coste. En operaciones con múltiples round-trips, subconsultas correlacionadas, paginación, el impacto es prohibitivo.

Cuello de botella en ancho de banda. Transferir millones de registros por conexiones intercontinentales puede convertir consultas de sub segundos en operaciones de minutos. En el caso real medido (100.000 filas, 5,40 MB), una conexión transatlántica a 100 Mbps tardaría 0,4 s de transferencia pura, pero la latencia de señalización por cada fragmento multiplicaria ese tiempo varias veces.

Costes de egress. En entornos cloud, extraer datos entre regiones incurre en costes de transferencia de red que a escala de terabytes pueden superar ampliamente el coste de cómputo del cluster Trino.

El predicate pushdown pasa de optimización a requisito critico. En el escenario local el pushdown mejora el rendimiento en un factor de 4,3x. En un escenario multi-región se vuelve imprescindible para la viabilidad de la consulta: sin él, se transportarán todos los clientes (1.000) y todas las ventas (100.000) para filtrar en destino. Con pushdown, solo viajan los 200 clientes españoles más sus ventas correspondientes.

Estrategia de JOIN según volumen relativo. El broadcast join actual (tabla pequeña -> todos los Workers) transfiere solo unas 200 filas por la WAN. Un partitioned join (shuffle de ambas tablas) generaría $N \times 100.000$ transmisiones entre regiones, haciendo la consulta inviable. En multi-región conviene forzar el broadcast explícitamente para la tabla de dimensiones.

Mitigaciones recomendables: desplegar un cluster Trino por región con federación cruzada solo para consultas tolerantes a latencia; replicar datos de referencia estáticos (catalogos, dimensiones) en cada region via S3 Cross-Region Replication; priorizar formatos columnares particionados para que la poda de ficheros del conector Hive evite transferencias innecesarias.

6. Reproducibilidad

Todo el código, configuración e infraestructura como código están versionados en el repositorio GitHub del grupo. El despliegue desde cero en cualquier máquina con Docker instalado se realiza en 4 pasos:

1. Clonar el repositorio
<code>git clone https://github.com/blasketch/big_data_a5.git && cd big_data_a5</code>
2. Generar datasets reproducibles (requiere Python 3, pandas, pyarrow)
<code>python script/generar_dataset.py</code>
3. Levantar el cluster Trino + PostgreSQL
<code>docker compose up -d</code>
4. Cargar clientes en PostgreSQL (primera vez o tras docker compose down -v)
<code>docker exec -it postgres-db psql -U usuario_trino -d base_datos_grupo \</code>
<code>-c "COPY clientes FROM '/docker-entrypoint-initdb.d/clientes_seed.csv' DELIMITER ','</code> <code>CSV HEADER;"</code>

Tras estos 4 pasos, la Trino Web UI es accesible en <http://localhost:8080> y la consulta federada puede ejecutarse desde cualquier cliente SQL o desde la CLI de Trino (`docker exec -it trino-coordinator trino`).

7. Conclusiones

El stack Trino 451 + PostgreSQL + conector Hive con metastore basado en filesystem demuestra que es posible montar un motor de consultas federadas funcional, ligero y portable con cuatro contenedores Docker y sin dependencias externas. La consulta federada implementada cruza dos catalogos heterogéneos (hive y postgresql), aplica funciones de agregación SUM, COUNT y AVG con GROUP BY, demuestra empíricamente el efecto del predicate pushdown (factor 4,3x de mejora en wall time medido en produccion), y permite analizar la distribución de splits y el plan de ejecución desde la interfaz web del Coordinator.

La decisión técnica más relevante fue sustituir PrestoDB por Trino. Ambos comparten arquitectura y sintaxis SQL, pero Trino aporta mejor soporte de plataforma, imágenes Docker actualizadas y una comunidad activa. La principal limitación del despliegue actual es el metastore Hive basado en filesystem: suficiente para esta práctica, pero que en un entorno productivo se sustituirá por un Hive Metastore real con backend PostgreSQL para gestionar esquemas evolutivos y control de concurrencia.

El proyecto queda en un estado deployable desde cero en cualquier máquina mediante git clone más 3 comandos.
